Requirements Gathering Document

Real-Time Traffic Analytics Using Azure

1. Purpose & Objective

Requirements gathering defines the goals, scope, and specific needs of the Real-Time Traffic Analytics project before active development begins. Following this structured framework minimizes scope creep, aligns the project team and stakeholders, and ensures the final delivery matches the expected technical and business outcomes.

2. The Requirements Gathering Process (Applied)

Step 1: Identify Stakeholders

We have mapped all parties impacted by or invested in the project to ensure alignment and manage expectations:

Internal Stakeholders:

Project Team: Amr Mahmoud (Lead/Architect), Mohamed Ibrahim (Ingestion), Mohsen Khalaf (QA), Mohamed Saleh (DB/dbt), Basant Magdy (Power BI), Kerolos Adly (DevOps).

DEPI Evaluators: Assessing technical implementation, documentation, and adherence to data engineering best practices.

External Stakeholders:

Digitraffic Finland API Provider: Supplies the source data and dictates API availability, schemas, and rate limits.

Dashboard End-Users: Transportation stakeholders or analysts who will review the unified traffic information via Power BI.

Step 2: Plan the Approach

Objectives & Scope: Build a 6-week, containerized, near-real-time data pipeline ingesting traffic, weather, and maintenance data, transforming it via dbt, and visualizing it in Power BI.

Elicitation Techniques:

Documentation Review: Deep dive into official Digitraffic API documentation to understand schemas, update frequencies, and limits.

Internal Collaborative Workshops: Team meetings to define the Kafka message contract, PostgreSQL schema, and dbt transformation logic.

Prototyping: Early Power BI mockups to align on dashboard visualization needs before final gold models are built.

Step 3: Conduct Elicitation

Through internal workshops and API analysis, we extracted the fundamental needs of the system, focusing on underlying data engineering challenges rather than premature UI details:

The system must handle mixed data formats (JSON and XML/DATEX II).

The pipeline must be resilient to temporary API failures without losing data context.

Data quality and deduplication must be enforced before data reaches the dashboard.

The environment must be fully reproducible using Docker.

Step 4: Document Observations and Findings

All requirements have been centralized and categorized into three distinct groups:

A. Functional Requirements (What the system must do):

Retrieve current data from selected Digitraffic API endpoints via scheduled Python scripts.

Publish parsed API payloads as events to designated Apache Kafka topics.

Consume Kafka messages and persist raw payloads with ingestion metadata into PostgreSQL.

Transform raw data into standardized Staging and analytics-ready Gold models using dbt Core.

Connect Power BI Desktop to the Gold layer to display interactive traffic, maintenance, and weather dashboards.

B. Non-Functional Requirements (How the system must perform):

Data Freshness: Target an end-to-end latency of 5–10 minutes where source cadence permits.

Reliability: Achieve ≥ 95% API ingestion success rate and ≥ 99% Kafka message processing success rate.

Reproducibility: The entire pipeline must be deployable via a single docker-compose command.

Security: No hardcoded credentials; all secrets must be managed via environment variables.

C. Project Constraints:

Time: Strict 6-week project timeline.

Infrastructure: Limited compute resources on a single Azure Virtual Machine.

External Dependency: Subject to Digitraffic API rate limits, uptime, and schema stability.

Step 5: Analyze and Prioritize

We applied the MoSCoW Prioritization Framework to rank capabilities based on technical feasibility, business value, and the 6-week timeline:

Must Have (MVP): Core Python ingestion for at least 2 key endpoints, Kafka streaming, PostgreSQL raw/gold storage, basic dbt transformations with uniqueness tests, local Docker setup, and a functional Power BI dashboard showing validated data.

Should Have: Ingestion of additional endpoints (e.g., Weather, Variable Signs), advanced error logging, and incremental dbt models.

Could Have: Automated Power BI Service refresh (dependent on licensing/connectivity) and advanced dashboard drill-through features.

Won't Have (Out of Scope): Predictive Machine Learning models, historical trend analysis over multiple years, and direct control of traffic infrastructure.

Step 6: Verify and Validate

To ensure absolute accuracy and completeness before active engineering:

Architecture Walkthrough: The team reviewed the data flow diagram to confirm all component interfaces (Python → Kafka → DB → dbt → Power BI) are clearly defined.

Schema Validation: Sample JSON/XML payloads were tested against the proposed PostgreSQL raw layer design.

Formal Sign-off: The team lead (Amr Mahmoud) and all module owners formally agreed on the "Minimum Viable Pipeline" scope and the MoSCoW prioritization before Week 3 development began.

Step 7: Manage Changes

To handle evolving scopes or mid-project discoveries gracefully:

Baseline Configuration: The approved Architecture Document and this Requirements Document serve as the project baseline.

Traceability: All tasks are logged in a shared project board (e.g., GitHub Projects/Trello), linking code commits and dbt models back to specific functional requirements.

Change-Control Process: Any request to add a new API endpoint, alter the Kafka message contract, or change the deployment architecture must be:

Proposed by the relevant module owner.

Reviewed for impact on timeline and Azure VM resources by the Team Lead.

Formally logged and approved before implementation begins.

3. Conclusion

By rigorously applying this 7-step requirements gathering process, the Real-Time Traffic Analytics project establishes a clear, agreed-upon foundation. This document serves as the single source of truth for the development team, ensuring that all technical efforts directly contribute to delivering a reliable, scalable, and well-documented data engineering solution.